Assignment 2B: Finding Spikes in Noisy Neural Data Cont.

Background

You are given a recording of continuous electrode voltage measurements. The recording contains neural spikes hidden among noise and slow drift in the baseline.

Your goal is to estimate the neuron's average firing rate during the recording.

Part 4: Implement and test the detector

What should you do?

Implement Part 3. Write code that detects spikes and returns their estimated sample indices or timestamps.

You may use Python and NumPy. Do not use a ready-made specialized spike detector that performs the main detection task for you.

Add tests to validate your code.

Ffor example, if you chose to use a threshold, your tests may include:

Test signal	Expected behavior
No samples above threshold	Detect zero events
One isolated sample above threshold	Detect one event
Several consecutive samples above threshold	Count one event, not several
Two separate above-threshold intervals	Detect two events
Signal starts above threshold	Correctly handle an event at the beginning

What should you submit?

- Your detection code.
- Your test code.
- A brief explanation of how the tests establish correctness.

Part 5: Estimate firing rate and evaluate sensitivity

What should you do?

Use your detector to estimate the number of events in the recording and calculate the average firing rate.

The rate is expressed in hertz (Hz) = spikes per second.

Investigate how your choice of parameters (e.g. threshold) affects the result.

What should you submit?

- Your primary firing-rate estimate.
- The comparison between different parameters.
- A plot showing the detected events on the voltage trace.
- A short interpretation of how stable your estimate is.
- A statement of the assumptions required to interpret this number as the neuron's firing rate during the recording.

Part 6 — Make NumPy earn its place

What should you do?

Now investigate whether you can perform the same detection operation more efficiently Implement the core event-detection operation in two ways:

Version A, Reference implementation

Write a straightforward implementation using a Python loop. Its main purpose is to provide a readable reference against which you can check correctness.

Version B, NumPy implementation

Write a vectorized implementation using NumPy arrays and Boolean operations to identify the beginnings of above-threshold intervals.

You should deliberately consider how to avoid a Python loop over every individual sample.

Use the same event definition, input signal, and threshold for both versions.

Then perform two evaluations.

Evaluation 1: Correctness

Check that both implementations return the same event indices. Explain why agreement between the two implementations provides evidence of correctness, and what it does not prove.

Evaluation 2: Runtime

Measure the runtime of the detection operation for each implementation.

Because a 10,000-sample signal is relatively small, runtime measurements may be noisy. Repeat each measurement several times and report a robust summary, such as the median runtime.

What should you submit?

- Both implementations.
- A correctness check comparing the outputs.
- The benchmark code.

A table containing the input size, timing method, median runtimes, and measured speedup.